

problem3

May 6, 2022

1 Problem 3 - Kubeflow, MiniKF, Kale 20 points

In this problem we will follow Kubeflow-Kale codelab ([link below](#)). You will follow the steps as outlined in the codelab to install Kubeflow with MiniKF, convert a Jupyter Notebook to Kubeflow Pipelines, and run Kubeflow Pipelines from inside a Notebook. For each step below you need to show the commands executed, terminal output, and screenshot of visual output (if any). You also need to give a new name to your GCP project and any resource instance you create, e.g., put your initial in the name string.

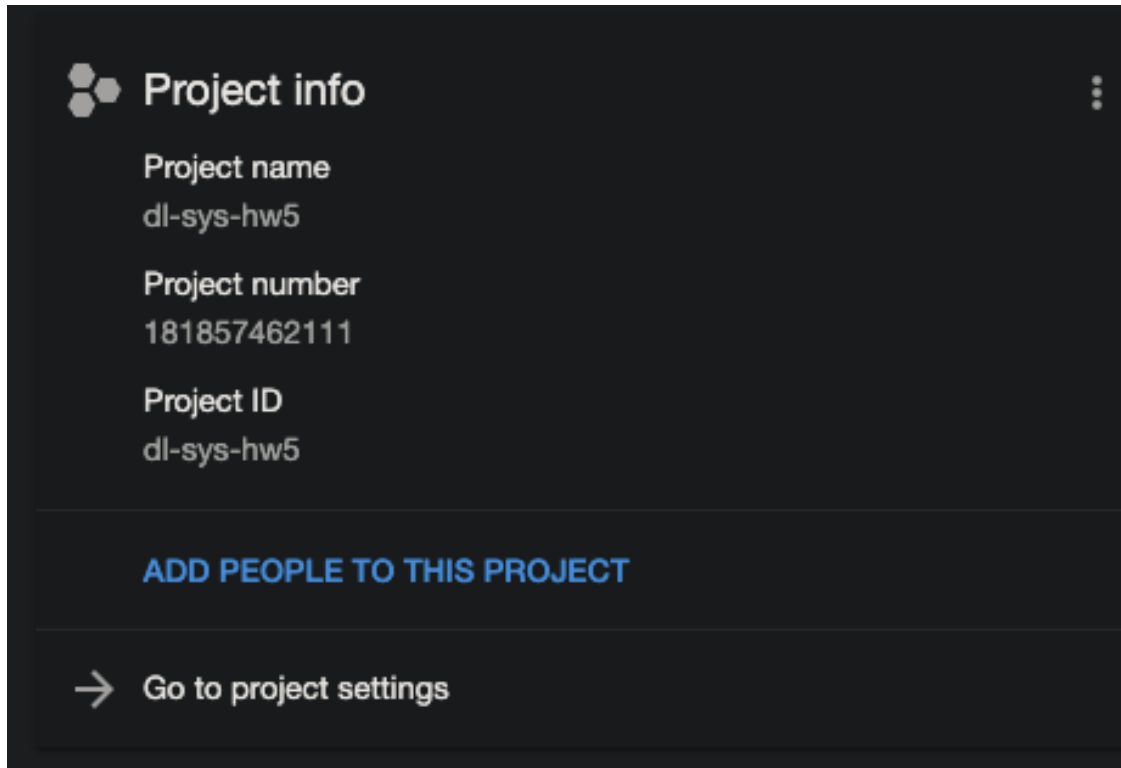
Problem 3 - Kubeflow, MiniKF, Kale

20 points

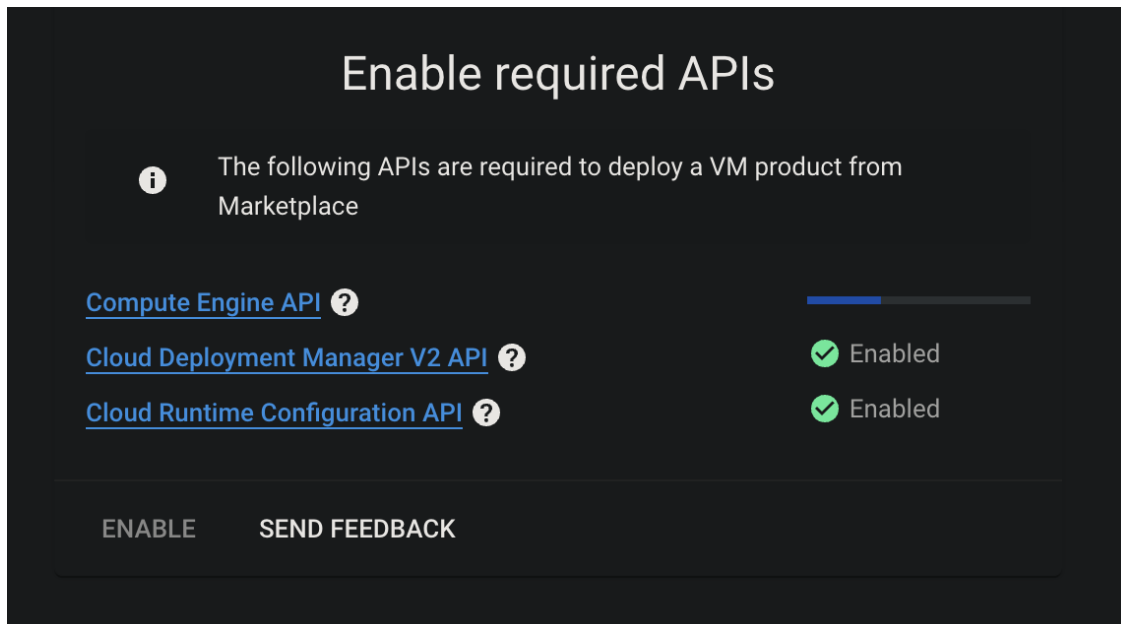
```
[1]: from IPython.display import Markdown, display  
  
display(Markdown("./problem3/3_1.md"))
```

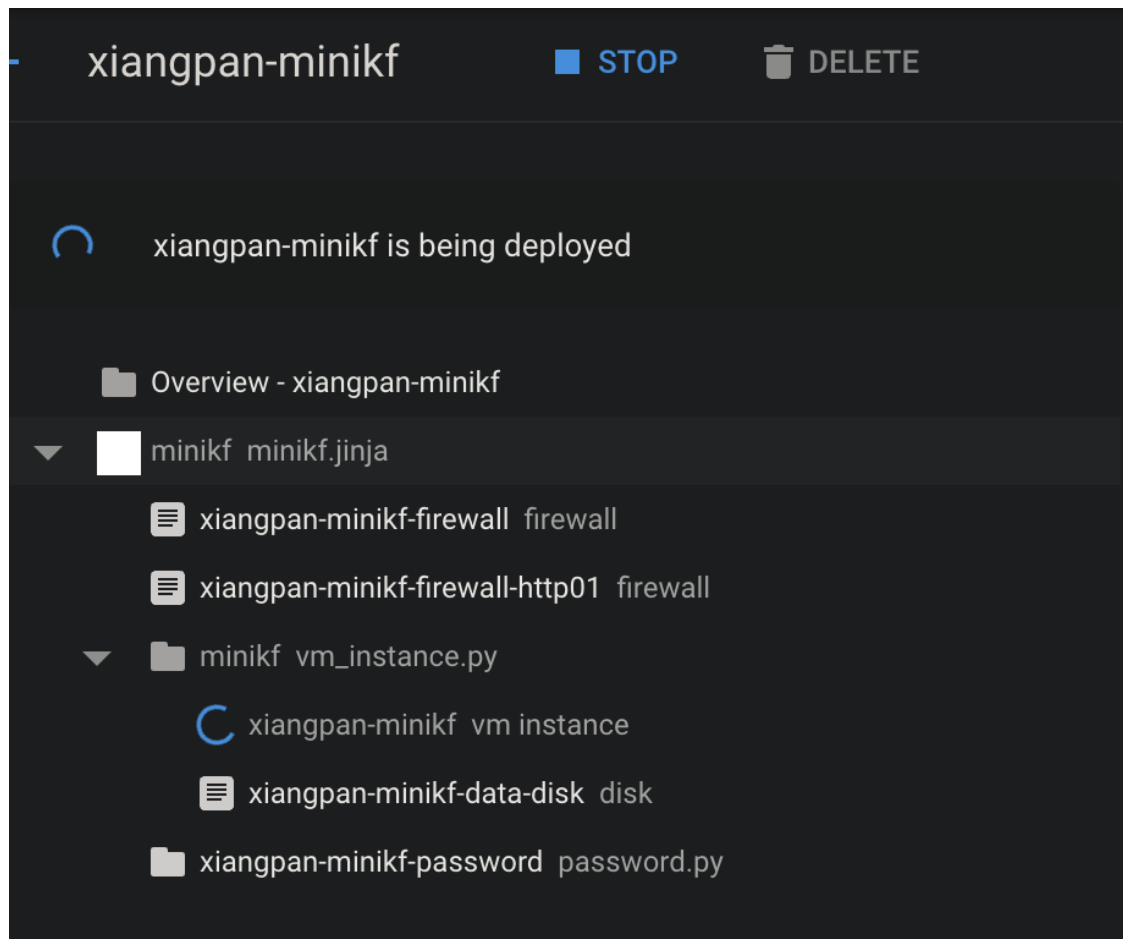
1.1 3-1

1.1.1 (a)



1.1.2 (b)

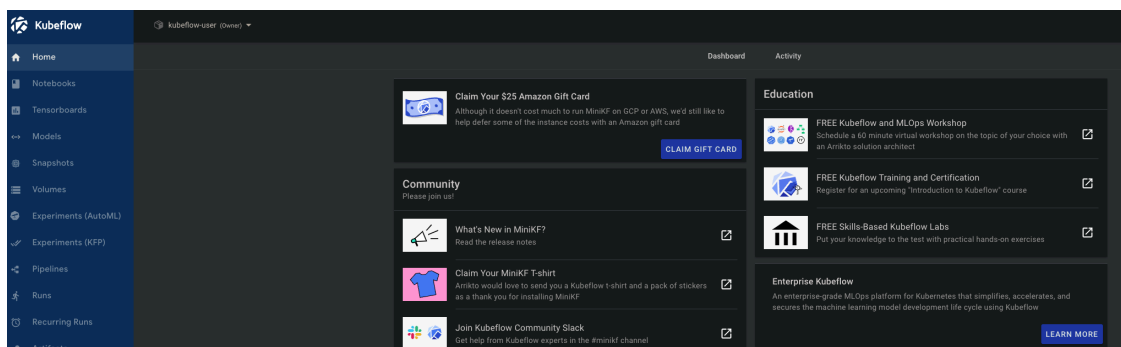
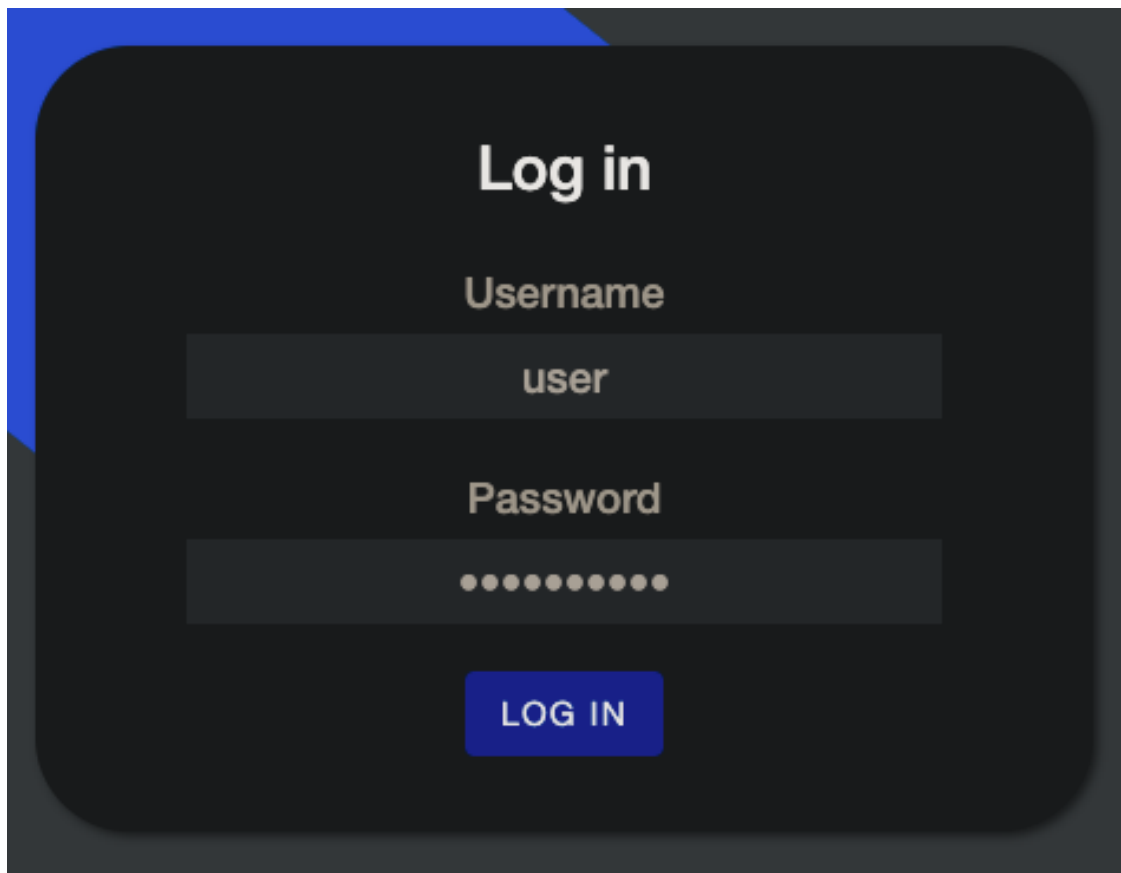
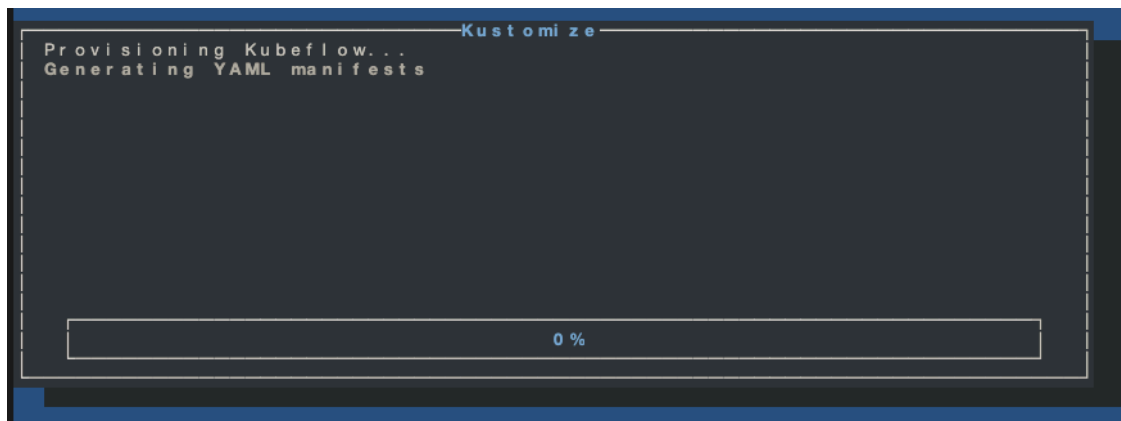




1.1.3 (c)

```
Welcome to MiniKF!  
Type "minikf" to ensure everything is up and running.  
mini kf@xiangpan-mini kf: ~$
```

```
Mini kube + Kube flow + Rok = Mini KF  
Provisioning completed.  
Log in to MiniKF here:  
https://xiangpan-mini kf.endpoints.dl-sys-hw5.cloud.goog  
Credentials:  
* Username: user  
* Password: wVwMg1Tzvs
```



1.2 3-2

1.2.1 (a)

Create a notebook server.



The screenshot shows the Kubeflow dashboard with a table of notebook servers. The table has columns for Status, Name, Type, Age, Last activity, Image, GPUs, CPUs, Memory, and Volumes. One server is listed: 'kubecon-xiangpan' with status 'Running', type 'JupyterLab', age 'less than a minute ago', last activity '-', image 'jupyter-kale-py36:releas...', 0 GPUs, 0.5 CPUs, 1Gi Memory, and 1 Volume. A 'CONNECT' button is next to it. Below the table is a terminal window showing the following output:

```
groups: cannot find name for group ID 1337
jovyan@kubecon-xiangpan-0:~$ git clone https://github.com/kubeflow-kale/kale
Cloning into 'kale'...
remote: Enumerating objects: 7064, done.
remote: Counting objects: 100% (695/695), done.
remote: Compressing objects: 100% (317/317), done.
remote: Total 7064 (delta 444), reused 512 (delta 371), pack-reused 6369
Receiving objects: 100% (7064/7064), 26.23 MiB | 20.58 MiB/s, done.
Resolving deltas: 100% (3700/3700), done.
jovyan@kubecon-xiangpan-0:~$
```

We need to install the pytorch and torchvision package.

Or we can choose the pytorch image from the supported list.

1.2.2 (b)

Download and run the notebook: We will be using pytorch-classification notebook from the example repo. Note that the code lab uses a different example from the repo (titanic dataset ml.ipynb). (3)

```
(base) jovyan@pytorch-xiangpan-0:~$ git clone https://github.com/kubeflow-kale/kale
Cloning into 'kale'...
remote: Enumerating objects: 7064, done.
remote: Counting objects: 100% (695/695), done.
remote: Compressing objects: 100% (318/318), done.
remote: Total 7064 (delta 443), reused 517 (delta 370), pack-reused 6369
Receiving objects: 100% (7064/7064), 26.23 MiB | 11.50 MiB/s, done.
Resolving deltas: 100% (3699/3699), done.
(base) jovyan@pytorch-xiangpan-0:~$
```

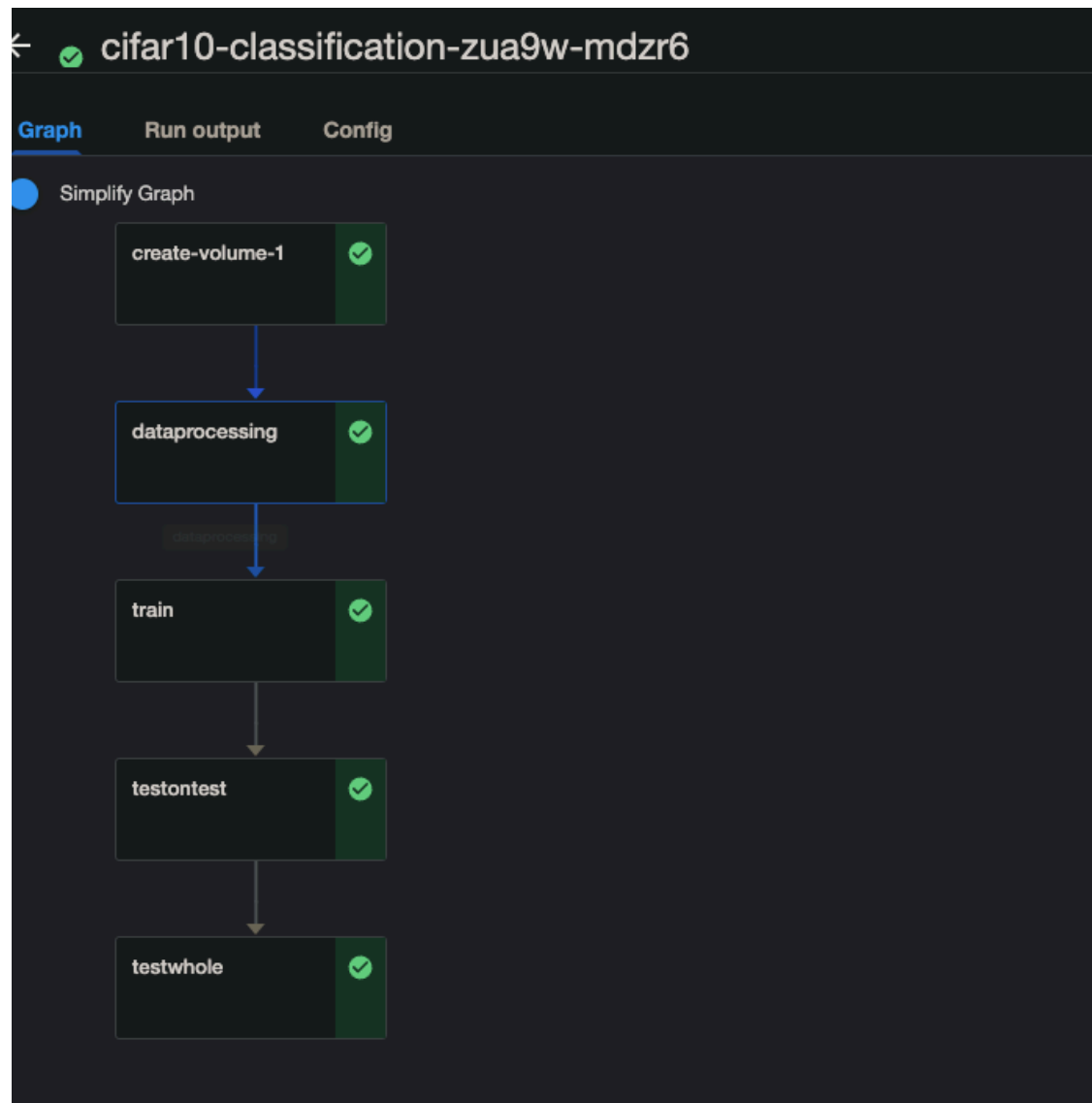
```
[11]: class_correct = list(0. for i in range(10))
      class_total = list(0. for i in range(10))
      with torch.no_grad():
          for data in testloader:
              images, labels = data
              outputs = net(images)
              _, predicted = torch.max(outputs, 1)
              c = (predicted == labels).squeeze()
              for i in range(4):
                  label = labels[i]
                  class_correct[label] += c[i].item()
                  class_total[label] += 1

      for i in range(10):
          print('Accuracy of %5s : %2d %%' % (
              classes[i], 100 * class_correct[i] / class_total[i]))
```

```
Accuracy of plane : 55 %
Accuracy of car : 75 %
Accuracy of bird : 38 %
Accuracy of cat : 41 %
Accuracy of deer : 51 %
Accuracy of dog : 42 %
Accuracy of frog : 74 %
Accuracy of horse : 45 %
Accuracy of ship : 72 %
Accuracy of truck : 50 %
```

1.2.3 (c) Compile

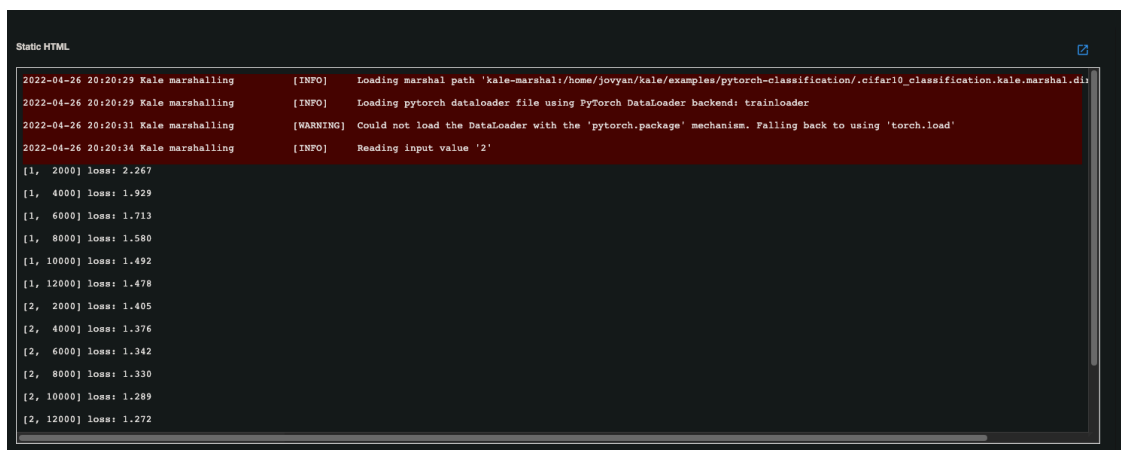
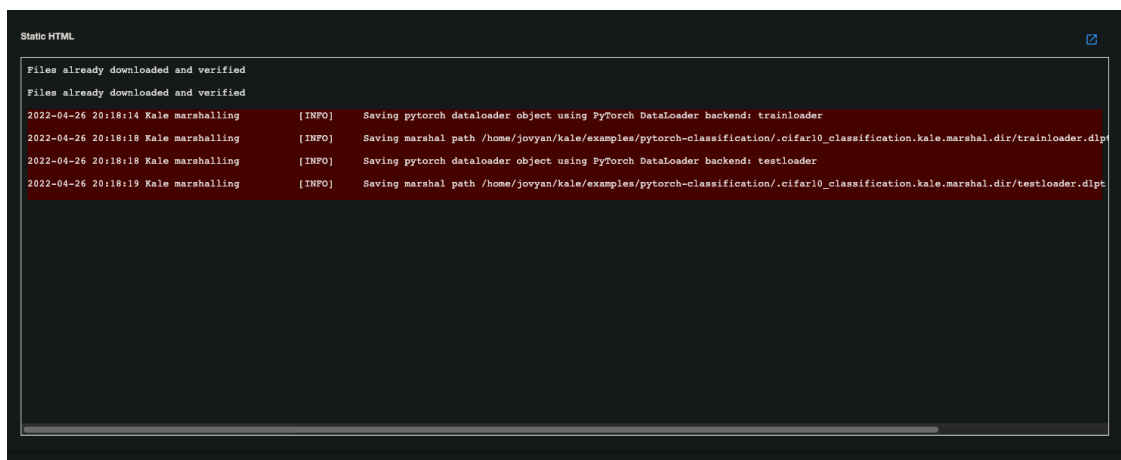
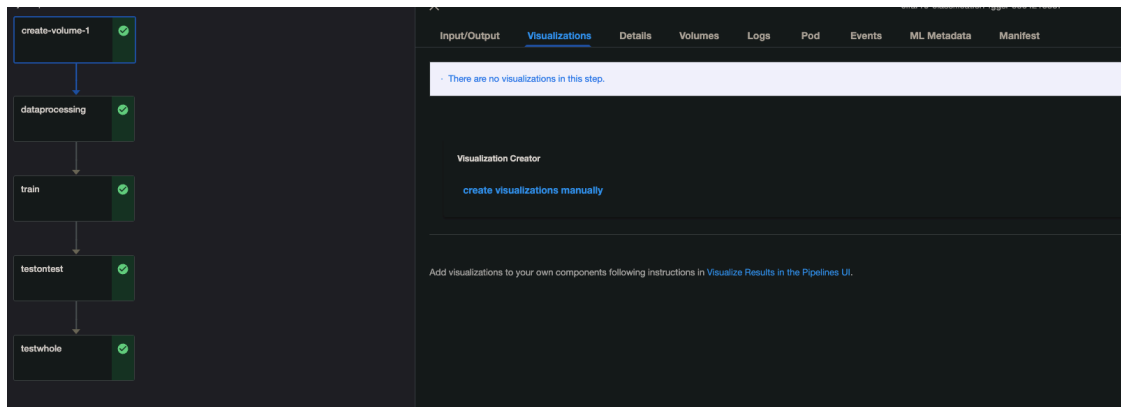
Convert your notebook to a KubeFlow Pipeline: Enable Kale and then compile and run the pipeline from Kale Deployment Panel. Show output from each of the 5 steps of the pipeline (5)



Graph **Run output** Config

No metrics found for this run.

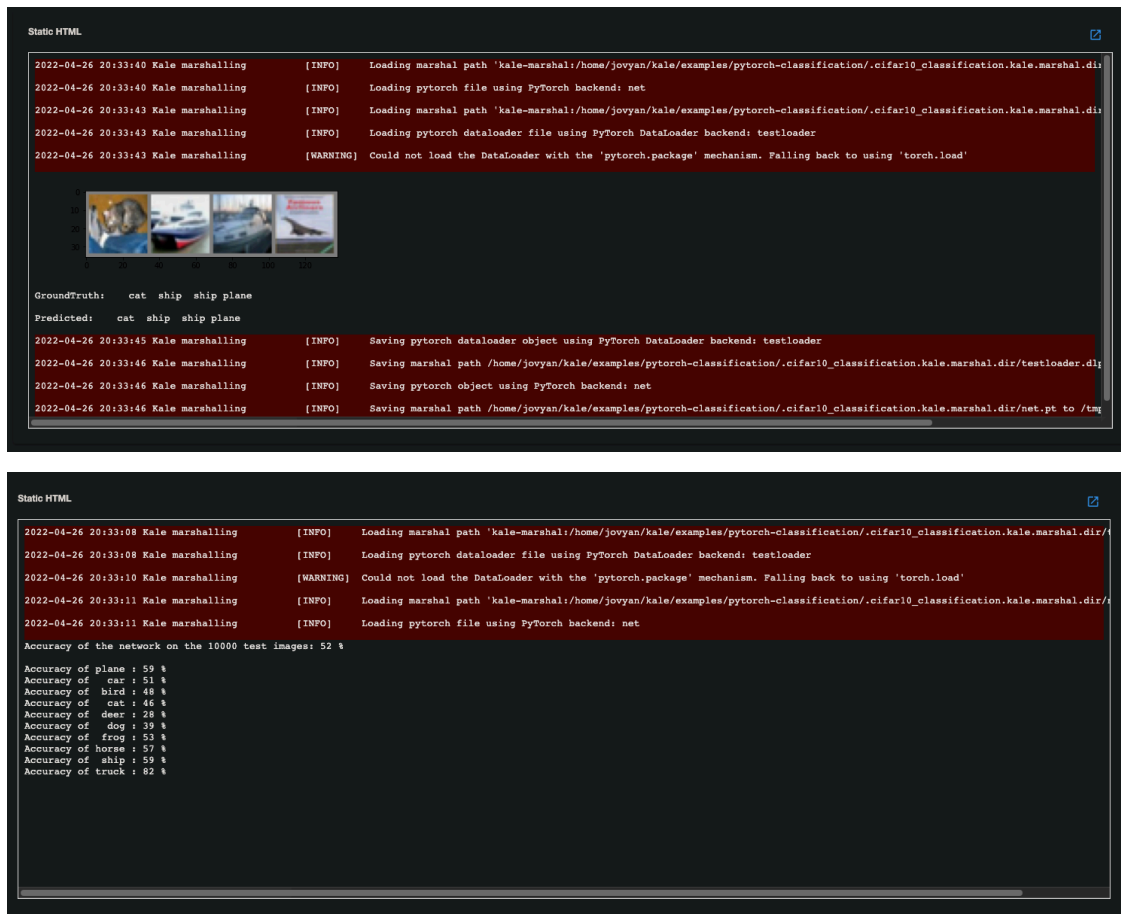
No output artifacts found for this run.



1.2.4 (d)

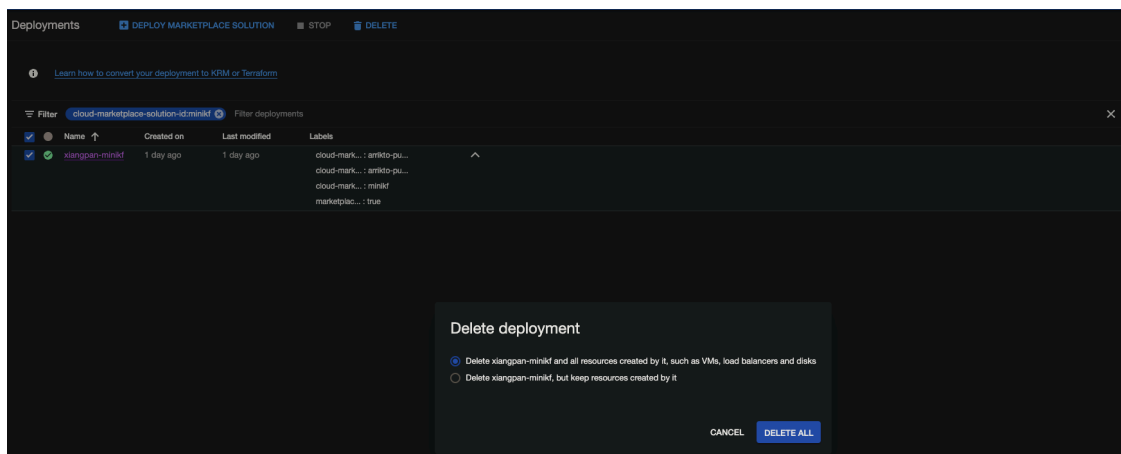
Show snapshots of "Graph" and "Run output" of the experiment. (3)

I'm not sure why I do not have the output of the in the run output section, but I have the output for each of the steps.



1.2.5 (e) Cleanup

Cleanup: Destroy the MiniKF VM



[]: